

- GLPK (GNU Linear Programming Kit), <http://www.gnu.org/software/glpk>. [12]
- Linear Programming Frequently Asked Questions, <http://www-unix.mcs.anl.gov/otc/Guide/faq/linear-programming-faq.html>. [13]
- Lustig, I.J., Marsten, R.E., and Shanno, D.F. 1994, "Interior Point Methods: Computational State of the Art," *ORSA Journal on Computing*, vol. 6, pp. 1–14. See also Commentaries and Rejoinder, pp. 15–36. [14]
- Wright, S.J. 1997, *Primal-Dual Interior-Point Methods* (Philadelphia: S.I.A.M.). [15]
- Bixby, R.E. 2002, "Solving Real-World Linear Programs: A Decade and More of Progress," *Operations Research*, vol. 50, pp. 3–15. [16]
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §4.10.
- Nemhauser, G.L., Rinnooy Kan, A.H.G., and Todd, M.J. (eds.) 1989, *Optimization* (Amsterdam: North-Holland).
- Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), Chapters 7–8.
- Ignizio, J.P., and Cavalier, T.M. 1994, *Linear Programming* (Englewood Cliffs, NJ: Prentice-Hall). [Undergraduate text]
- Vanderbei, R.J. 2001, *Linear Programming: Foundations and Extensions*, 2nd ed. (Boston: Kluwer). [Undergraduate/graduate text]
- Chvátal, V. 1983, *Linear Programming* (New York: Freeman). [Undergraduate text]
- Gass, S.I. 1985, *Linear Programming: Methods and Applications*, 5th ed. (New York: McGraw-Hill), reprinted 2003 (New York: Dover). [Undergraduate text]
- Murty, K.G. 1983, *Linear Programming* (New York: Wiley). [Undergraduate text]
- Numerical Recipes Software 2007, "Routine Implementing the Simplex Method," *Numerical Recipes Webnote No. 12*, at <http://www.nr.com/webnotes?12> [17]

10.11 Linear Programming: Interior-Point Methods

As we mentioned in §10.10, the worst-case number of iterations for the simplex method is an exponential function of n . (The worst case occurs for $m = n$.) The average number of iterations, however, is a small multiple of m . For a long time it was not known if there was another algorithm for linear programming that would be bounded by, for example, some polynomial in n . In 1979, Khachian published a new algorithm [1], the *ellipsoid method*, that is in fact polynomial in n . Disappointingly, however, in practical implementations it was much slower than the simplex method.

In 1984, the field was electrified by Karmarkar's paper [2] describing an *interior-point* method. Not only was it polynomial in n , but he claimed it solved large LP problems significantly faster than the simplex method. This claim turned out to be somewhat exaggerated, but in the frenzy of activity over the next decade interior-point algorithms were developed that *do* solve many problems much faster than the simplex method, especially very large problems. Ironically, the rivalry between the two algorithms led to improvements of about a factor of 100 in the simplex method itself over the same period. We give some recommendations on which method to use at the end of this section.

Originally, interior-point methods traversed the interior of the feasible region, homing in on the optimal vertex. So-called infeasible interior-point methods follow

a path in the interior of the nonnegative region, $x_i \geq 0$, $i = 1, \dots, n$, but possibly through the infeasible region.

To understand how interior-point methods work, we need to develop some more theory about linear programming, in particular about *duality*.

10.11.1 Dual Problem

As we saw in §10.10, any LP problem can be written in standard form:

$$\begin{array}{ll} \text{minimize} & \mathbf{c} \cdot \mathbf{x} \\ \text{subject to} & \mathbf{A} \cdot \mathbf{x} = \mathbf{b} \\ & \mathbf{x} \geq 0 \end{array} \quad (10.11.1)$$

Here slack variables have been appended to the x 's to write all inequality constraints as equalities, but no other logical variables have been added. This is called the *primal* problem. Recall that if the constraints in (10.11.1) are satisfied, we say that $\mathbf{x}$ is a feasible point.

The *dual* problem corresponding to (10.11.1) interchanges the roles of variables and constraints: Corresponding to the m constraints is a set of variables $(y_1, \dots, y_m)$ determined by

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{c} \\ & \mathbf{y} \text{ free} \end{array} \quad (10.11.2)$$

Here “free” means unconstrained. Most of the textbooks mentioned at the end of the previous section discuss exactly how to go from the primal problem to its dual. For a hint, see §16.5.2, where a different primal-dual problem is discussed. After that, a particularly clear discussion is in [3]. Note how the constraint matrix for the dual problem is the transpose of the matrix for the primal problem. Forming the dual of the dual simply takes you back to the primal problem.

The dual problem (10.11.2) can be rewritten by adding slack variables $(z_1, \dots, z_n)$:

$$\begin{array}{ll} \text{maximize} & \mathbf{b} \cdot \mathbf{y} \\ \text{subject to} & \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} = \mathbf{c} \\ & \mathbf{z} \geq 0, \quad \mathbf{y} \text{ free} \end{array} \quad (10.11.3)$$

If $(\mathbf{y}, \mathbf{z})$ satisfy the constraints in (10.11.3), we say that they are *dual feasible*. For a good introduction to the meaning of the relation between primal and dual problems, see [4].

The *weak duality theorem* asserts that the value of the dual objective function provides a lower bound to the value of the objective function if they are each evaluated at feasible points. Proof: $\mathbf{b} \cdot \mathbf{y} = \mathbf{y} \cdot \mathbf{A} \cdot \mathbf{x} = \mathbf{x} \cdot \mathbf{A}^T \cdot \mathbf{y} \leq \mathbf{x} \cdot \mathbf{c}$. The difference

$$\mathbf{c} \cdot \mathbf{x} - \mathbf{b} \cdot \mathbf{y} \quad (10.11.4)$$

is called the *duality gap*. If the primal is unbounded (the objective can be made arbitrarily negative), then the dual must be infeasible, and vice versa. Moreover, we

have the *strong duality theorem*: If either the primal or the dual has a finite optimal solution, so does the other, and $\mathbf{c} \cdot \mathbf{x} = \mathbf{b} \cdot \mathbf{y}$ for the optimal solution.

There is a further important relation between the primal and dual variables at optimality. Consider a particular x_j and the corresponding $z_j = (\mathbf{c} - \mathbf{A}^T \cdot \mathbf{y})_j$. The *Karush-Kuhn-Tucker complementarity condition* says that they can't both be strictly greater than zero: At least one must be equal to zero. In other words,

$$x_j z_j = 0, \quad j = 1, \dots, n \quad (10.11.5)$$

Adopting the convention that an uppercase letter denotes a matrix with the corresponding lowercase vector along the diagonal, we can write equation (10.11.5) alternatively as

$$\mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} = 0, \quad \mathbf{X} = \text{diag}(x_1, \dots, x_n), \quad \mathbf{Z} = \text{diag}(z_1, \dots, z_n), \quad \mathbf{e} = (1, \dots, 1) \quad (10.11.6)$$

Since each x_j and z_j is nonnegative, equation (10.11.5) is equivalent to $\mathbf{x} \cdot \mathbf{z} = 0$. In fact, this result holds in both directions: The *complementary slackness theorem* says that feasible solutions are optimal if and only if $\mathbf{x} \cdot \mathbf{z} = 0$. It is easy to show that for feasible solutions, $\mathbf{x} \cdot \mathbf{z}$ is simply equal to the duality gap (10.11.4).

Note that complementary slackness allows the possibility that both x_j and z_j might be zero at an optimal solution. *Strict complementarity* is the property that exactly one of these quantities is zero for all j . The *Goldman-Tucker theorem* says that if the primal and dual are feasible, there exists a strictly complementary pair of optimal solutions. As we'll see, interior-point methods find such a solution.

10.11.2 The KKT Conditions

Linear programming is a special case of general constrained optimization, where one wants to minimize some function $f(\mathbf{x})$ subject to constraints. The general optimality conditions are called the Karush-Kuhn-Tucker or KKT conditions. Specialized to the LP problem (10.11.1), the KKT conditions are

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} & \mathbf{x} &\geq 0 \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} &= \mathbf{c} & \mathbf{z} &\geq 0 \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= 0 & \mathbf{y} &\text{ free} \end{aligned} \quad (10.11.7)$$

Note that these are exactly the conditions that follow from strong duality and complementarity. Later we will see how to derive these conditions directly using Lagrange multipliers to handle the constraints.

The KKT conditions (10.11.7) are necessary and sufficient for $\mathbf{x}$ to be an optimal solution of (10.11.1). Moreover, they are necessary and sufficient for $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ to solve the primal and dual problems (10.11.1) and (10.11.2). In this case, we call $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ a *primal-dual solution*. Primal-dual interior-point methods solve the equations (10.11.7) in such a way that the inequalities are satisfied strictly at every iteration, that is, $\mathbf{x}, \mathbf{z} > 0$.

The equations are solved using a variant of Newton's method. Recall that if we define the vector of equations in (10.11.7) as

$$\mathbf{F}(\mathbf{v}) = \begin{bmatrix} \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} = 0 \quad (10.11.8)$$

where $\mathbf{v}$ is shorthand for $(\mathbf{x}, \mathbf{y}, \mathbf{z})$, then Newton's method determines the update $\Delta \mathbf{v}$ to the current point by solving

$$\mathbf{J} \cdot \Delta \mathbf{v} = -\mathbf{F} \quad (10.11.9)$$

Here $\mathbf{J}$ is the Jacobian matrix of $\mathbf{F}$ (see §9.7). A full step with this value of $\Delta \mathbf{v}$ is usually not allowed because it would violate the condition $(\mathbf{x}, \mathbf{z}) \geq 0$. So the new iterate is chosen from a line search along the Newton direction:

$$\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{old}} + \alpha \Delta \mathbf{v}, \quad \alpha \in (0, 1] \quad (10.11.10)$$

You choose $\alpha = 1$ if possible; otherwise, you choose the maximum α that preserves nonnegativity.

Note the importance of keeping the nonnegative variables strictly positive at all times: The Newton equation for $x_j z_j = 0$ is $x_j \Delta z_j + z_j \Delta x_j = -x_j z_j$. Suppose z_j is zero. Then the Newton equation becomes $x_j \Delta z_j = 0$, or $\Delta z_j = 0$. So z_j remains zero if it ever becomes zero. The algorithm can never recover. Of course, one should also expect difficulties if any variable gets “too close” to zero.

This simple damped Newton's method is not a practical algorithm, because too often the allowed stepsize becomes very small ($\alpha \ll 1$). There are two important modifications that are crucial to producing a viable algorithm:

- Change the search direction so that it aims toward the “center” of the nonnegative region. The idea is to allow larger steps before one of the variables would become negative.
- Don't allow the variables to come “too close” to the boundary of the nonnegative region. As discussed above, little progress tends to be made from such points.

10.11.3 The Central Path

One way to bias the search direction away from boundary is to arrange for all the complementarity pairs $x_j z_j$ to converge to zero at the same rate, say $x_j z_j = \tau$, where $\tau \rightarrow 0$ during the iterations. In other words, modify the last equation in (10.11.8) so that the system becomes

$$\mathbf{F}(\mathbf{v}) = \begin{bmatrix} \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \tau \mathbf{e} \end{bmatrix} \quad (10.11.11)$$

The set of solutions $\mathbf{v}(\tau)$ to equations (10.11.11) defines the *central path*. Primal-dual algorithms take steps toward points on the central path with $\tau > 0$. During the iterations, $\tau \rightarrow 0$ and the central path homes in on the optimal solution.

If you plot the central path in the hyperspace of $(\mathbf{x}, \mathbf{z})$ coordinates, it's some contorted line that doesn't look central to anything. However, if you plot it in coordinates $(x_1 z_1, x_2 z_2, \dots)$, you see that it is equidistant from all the coordinate surfaces, with the optimal solution at the origin. When the current iterate is close to the central path, the next iteration can make a large step toward the optimal solution. When the current iterate is close to one of the boundaries, a good algorithm makes the next iteration get close to the central path again.

10.11.4 Path-Following Methods

Path-following methods don't just aim steps in the direction of the central path; they explicitly attempt to stay close to it. These methods are currently the most successful interior-point methods. In primal-dual methods, the duality gap (10.11.4), which is equal to $\mathbf{x} \cdot \mathbf{z}$ for feasible points, provides a figure-of-merit for how close one is to the optimal solution. Accordingly, we set

$$\mu = \mathbf{x} \cdot \mathbf{z} / n, \quad \tau = \mu \delta, \quad \delta \in [0, 1] \quad (10.11.12)$$

The quantity δ is called the *centering parameter*, while μ is called the *duality measure*. If $\delta = 1$, the Newton step calculated from (10.11.11) is in a *centering direction*, toward a point at which each product $x_j z_j$ is equal to the average value μ defined in (10.11.12). On the other hand, the value $\delta = 0$ defines the Newton step for the original system (10.11.8). Good algorithms use intermediate values to trade off between improving centrality and reducing μ .

Methods that keep δ close to 1, so that unit steps ($\alpha = 1$) stay close to the central path, are called *short-step* methods. Methods that allow small values of δ are called *long-step* methods (less conservative choices of δ). There is an interesting gap between theory and practice between the methods. Short-step methods have been proved to converge in $O(\sqrt{n} \log \frac{1}{\epsilon})$ iterations, where ϵ is the desired tolerance. Long-step methods take $O(n \log \frac{1}{\epsilon})$ iterations, according to theory. Yet in practice short-step methods take hopelessly small steps, while long-step methods provide practical algorithms.

This is a somewhat academic discussion anyway. Real-life examples take many fewer than $O(\sqrt{n})$ iterations — a few dozen is typical for large problems.

10.11.5 Barrier Methods

Introducing a “penalty” function is a standard technique to enforce a constraint in general optimization problems. For example, to enforce the condition $\mathbf{x} \geq 0$, consider the logarithmic penalty function

$$\sum_{j=1}^n \log x_j \quad (10.11.13)$$

If any $x_j \rightarrow 0$, this function tends to $-\infty$. So instead of trying to minimize $\mathbf{c} \cdot \mathbf{x}$ in the standard primal problem (10.11.1), consider minimizing

$$\mathbf{c} \cdot \mathbf{x} - \tau \sum_{j=1}^n \log x_j \quad (10.11.14)$$

If one takes the limit $\tau \rightarrow 0$ after the minimization, we expect this to be equivalent to the original problem.

Equation (10.11.14) is called a *logarithmic barrier function*. It defines a family of nonlinear objective functions that gives the solution to the original problem as the parameter $\tau \rightarrow 0$.

The power of the barrier function idea is that it lets us handle the constraint $\mathbf{x} \geq 0$ with calculus. To minimize (10.11.14) subject to the constraint $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$, introduce a Lagrange multiplier $-\mathbf{y}$ and extremize the Lagrangian

$$L = \mathbf{c} \cdot \mathbf{x} - \tau \sum_{j=1}^n \log x_j - \mathbf{y} \cdot (\mathbf{A} \cdot \mathbf{x} - \mathbf{b}) \quad (10.11.15)$$

The optimality conditions $\nabla_x L = 0$ and $\nabla_y L = 0$ give

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \tau \mathbf{X}^{-1} \cdot \mathbf{e} &= \mathbf{c} \end{aligned} \quad (10.11.16)$$

Define the vector

$$\mathbf{z} = \tau \mathbf{X}^{-1} \cdot \mathbf{e}, \quad \text{i.e.,} \quad z_j = \tau/x_j \quad (10.11.17)$$

Then equation (10.11.16) becomes

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} &= \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= \tau \mathbf{e} \end{aligned} \quad (10.11.18)$$

These are exactly the equations (10.11.11) defining the central path, and they reduce to the KKT conditions (10.11.7) if we set τ to zero.

Note that equation (10.11.16) can be used to define an algorithm, the *primal interior-point method*, that doesn't depend on $\mathbf{z}$. Similarly, by starting with a logarithmic barrier function for the dual objective function, one can derive a purely dual method that doesn't involve $\mathbf{x}$. In practice, These methods are not competitive with the primal-dual methods.

Originally the logarithmic barrier function idea played an important role in motivating interior-point methods. More recently, the viewpoint has shifted to emphasize the importance of τ as defining the centering property of the algorithm rather than being simply a parameter to enforce the nonnegativity constraint.

10.11.6 A Primal-Dual Infeasible Interior-Point Algorithm

Let's pull all the pieces together now to define the algorithm. Write equation (10.11.11) for the new iterate:

$$\begin{aligned} \mathbf{A} \cdot (\mathbf{x} + \Delta \mathbf{x}) - \mathbf{b} &= 0 \\ \mathbf{A}^T \cdot (\mathbf{y} + \Delta \mathbf{y}) + \mathbf{z} + \Delta \mathbf{z} - \mathbf{c} &= 0 \\ (\mathbf{X} + \Delta \mathbf{X}) \cdot (\mathbf{Z} + \Delta \mathbf{Z}) \cdot \mathbf{e} &= \tau \mathbf{e} \end{aligned} \quad (10.11.19)$$

Drop the quadratic term $\Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e}$ and get

$$\begin{bmatrix} \mathbf{A} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}^T & \mathbf{1} \\ \mathbf{Z} & \mathbf{0} & \mathbf{X} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x} \\ \Delta \mathbf{y} \\ \Delta \mathbf{z} \end{bmatrix} = \begin{bmatrix} -\mathbf{r}_p \\ -\mathbf{r}_d \\ \tau \mathbf{e} - \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} \quad (10.11.20)$$

where the primal and dual residuals are defined by

$$\begin{aligned} \mathbf{r}_p &= \mathbf{A} \cdot \mathbf{x} - \mathbf{b} \\ \mathbf{r}_d &= \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{c} \end{aligned} \quad (10.11.21)$$

Equation (10.11.20) is simply the Newton equation (10.11.9) for (10.11.11). Note that the only nonlinearity comes from the innocuous looking quadratic term for complementary slackness. Yet it's exactly what leads to all the difficulty!

Since $\mathbf{X}$ is a diagonal positive-definite matrix, we can trivially invert it and use the last equation in (10.11.20) to eliminate $\Delta \mathbf{z}$ from the second equation. Interchanging the order of the variables $\Delta \mathbf{x}$ and $\Delta \mathbf{y}$, we get

$$\begin{bmatrix} \mathbf{0} & \mathbf{A} \\ \mathbf{A}^T & -\mathbf{X}^{-1} \cdot \mathbf{Z} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{y} \\ \Delta \mathbf{x} \end{bmatrix} = \begin{bmatrix} -\mathbf{r}_p \\ \mathbf{z} - \tau \mathbf{X}^{-1} \cdot \mathbf{e} - \mathbf{r}_d \end{bmatrix} \quad (10.11.22)$$

Similarly, since $-\mathbf{X}^{-1} \cdot \mathbf{Z}$ is easily invertible, we can use the second equation in (10.11.22) to eliminate $\Delta \mathbf{x}$ from the first. This gives

$$\mathbf{A} \cdot (\mathbf{X} \cdot \mathbf{Z}^{-1}) \cdot \mathbf{A}^T \cdot \Delta \mathbf{y} = -\mathbf{r}_p + \mathbf{A} \cdot (\mathbf{x} - \tau \mathbf{Z}^{-1} \cdot \mathbf{e} - \mathbf{X} \cdot \mathbf{Z}^{-1} \cdot \mathbf{r}_d) \quad (10.11.23)$$

These are called the *normal equations*, by analogy with the normal equations that occur in least-squares problems (cf. 15.4.10). The predecessor equations in (10.11.22) are called the *augmented equations*. Note that the matrix on the left-hand side of the normal equations is symmetric and positive-definite, except for some delicacy as $\mathbf{x}$ and $\mathbf{z} \rightarrow 0$. This suggests solving them with some version of the Cholesky decomposition (§2.9).

Once $\Delta \mathbf{y}$ is determined from the normal equations, the second equation in (10.11.20) gives $\Delta \mathbf{z}$:

$$\Delta \mathbf{z} = -\mathbf{A}^T \cdot \Delta \mathbf{y} - \mathbf{r}_d \quad (10.11.24)$$

Finally, the third equation in (10.11.20) gives $\Delta \mathbf{x}$:

$$\Delta \mathbf{x} = -\mathbf{X} \cdot \mathbf{Z}^{-1} \cdot \Delta \mathbf{z} + \tau \mathbf{Z}^{-1} \cdot \mathbf{e} - \mathbf{x} \quad (10.11.25)$$

In a feasible interior-point method, an initial point is somehow found in the feasible region, that is, with $\mathbf{r}_p = \mathbf{r}_d = 0$ and $(\mathbf{x}, \mathbf{z}) > 0$. Then equations (10.11.23) – (10.11.25) are solved with $\mathbf{r}_p$ and $\mathbf{r}_d$ set to zero. The consensus now is that it is not necessary to do this. It is much easier to choose a point that may be infeasible initially, and allow the iterations to converge toward a feasible point. As explained above, it is still crucial to maintain nonnegativity, however.

Equation (10.11.23) contains three contributions to the step $\Delta \mathbf{y}$ and hence to $\Delta \mathbf{x}$ and $\Delta \mathbf{z}$. First there are the terms that involve $\mathbf{r}_p$ and $\mathbf{r}_d$, which drive the solution toward feasibility. Then there is the term independent of τ . It drives the solution toward optimality. In the literature, this term is called the *affine scaling* term, because there is a geometric interpretation of its effect in terms of a linear scaling of variables. Finally there is the term proportional to τ , which is the centering term.

Here is the framework for a simple primal-dual infeasible interior-point method:

1. Choose an initial nonnegative point.
2. If the infeasibilities $\mathbf{r}_p$ and $\mathbf{r}_d$ and the complementarity gap $\mathbf{x} \cdot \mathbf{z}$ are below the desired tolerance, exit. Otherwise, continue.
3. Set the value of τ from equation (10.11.12). A value of $\delta \approx 0.02$ works well.
4. Compute the direction of the step $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{z})$ from equations (10.11.23) – (10.11.25). The solution of the normal equations is done in two steps: factorization of the matrix to some easily invertible form, followed by solution using this factorization.
5. Determine the maximum stepsizes that do not allow the variables to become negative. Separate stepsizes can be determined for the primal and dual variables:

$$\begin{aligned}
\mathbf{x}_{\text{new}} &= \mathbf{x}_{\text{old}} + \alpha_p \Delta \mathbf{x} \\
\mathbf{y}_{\text{new}} &= \mathbf{y}_{\text{old}} + \alpha_d \Delta \mathbf{y} \\
\mathbf{z}_{\text{new}} &= \mathbf{z}_{\text{old}} + \alpha_d \Delta \mathbf{z}
\end{aligned}
\tag{10.11.26}$$

where α_p and α_d are initially chosen to be the largest values that keep all components of $\mathbf{x}_{\text{new}}$ and $\mathbf{z}_{\text{new}}$ nonnegative but no larger than unity. Then reduce the values of α_p and α_d by a safety factor σ . A conservative choice is $\sigma = 0.9$, but $\sigma = 0.99995$ works for many problems.

6. Go back to step 2 for the next iteration.

Since in real-life linear programming the constraint matrix $\mathbf{A}$ is sparse, the code must take advantage of this. The various matrix products such as $\mathbf{A} \cdot \mathbf{x}$, $\mathbf{A}^T \cdot \mathbf{y}$ and $\mathbf{A} \cdot (\mathbf{X} \cdot \mathbf{Z}^{-1}) \cdot \mathbf{A}^T$ should be computed efficiently. More important, the factorization and backsubstitution involved in solving the normal equations must use a suitable sparse matrix Cholesky decomposition. The factorization step in fact dominates the running time of the algorithm. Our implementation uses the relatively simple package LDL [5], combined with the package AMD [6] to compute an ordering (permutation) of the matrix that minimizes fill-in during the factorization. Both of these packages are freely available. Note that LDL has to be modified to deal with the singularities that occur as the diagonal matrix elements $x_j/z_j \rightarrow 0$. It is sufficient to modify the line of code in LDL that tests for a diagonal element equal to zero to something like

```
if (D[k] < 1.0e-40)
    D[k] = 1.0e128;
```

This has the effect of setting the corresponding variable to zero, which is the desired behavior. Here is our interface NR1dl.h to these packages. The full implementation is given in a Webnote [13].

interior.h

```
extern "C" {
    #include "ldl.h"
    #include "amd.h"
}

struct NR1dl {
Interface between Numerical Recipes routine intpt and the required external packages LDL and AMD.
    Doub Info [AMD_INFO];
    Int lnz,n,nz;
    VecInt PP,PPinv,PPattern,LLnz,LLp,PParent,FFlag,*LLi;
    VecDoub YY,DD,*LLx;
    Doub *Ax, *Lx, *B, *D, *X, *Y;
    Int *Ai, *Ap, *Li, *Lp, *P, *Pinv, *Flag,*Pattern, *Lnz, *Parent;
    NR1dl(NRsparseMat &adat);
    Constructor only needs adat to have been declared with appropriate dimensions.
    void order();
    AMD ordering and LDL symbolic factorization. Only needs nonzero pattern of adat, not actual values.
    void factorize();
    Numerical factorization of matrix.
    void solve(VecDoub_O &y,VecDoub &rhs);
    Solves for y given rhs. Can be invoked multiple times after a single call to factorize.
    ~NR1dl();
};
```

Here is a simple implementation of the interior-point algorithm. Although it is a pedagogical code, it is actually quite powerful — better than the pedagogical

simplex code of the previous section. Below we will explain what would be required to turn this code into a serious implementation.

Doub dotprod(VecDoub_I &x, VecDoub_I &y)

interior.h

Compute the dot product of two vectors, $\mathbf{x} \cdot \mathbf{y}$.

```
{
    Doub sum=0.0;
    for (Int i=0;i<x.size();i++)
        sum += x[i]*y[i];
    return sum;
}
```

Int intpt(const NRsparseMat &a, VecDoub_I &b, VecDoub_I &c, VecDoub_O &x)

Interior-point method for linear programming. On input a contains the coefficient matrix for the constraints in the form $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$. The right-hand side of the constraints is input in $\mathbf{b}[0..m-1]$. The coefficients of the objective function to be minimized, $\mathbf{c} \cdot \mathbf{x}$, are input in $\mathbf{c}[0..n-1]$. Note that $\mathbf{c}$ should generally be padded with zeros corresponding to the slack variables that extend the number of columns to be n . The function returns 0 if an optimal solution is found; 1 if the problem is infeasible; 2 if the dual problem is infeasible, i.e., if the problem is unbounded or perhaps infeasible; and 3 if the number of iterations is exceeded. The solution is returned in $\mathbf{x}[0..n-1]$.

```
{
    const Int MAXITS=200;           Maximum iterations.
    const Doub EPS=1.0e-6;         Tolerance for optimality and feasibility.
    const Doub SIGMA=0.9;          Stepsize reduction factor (conservative choice).
    const Doub DELTA=0.02;         Factor to set centrality parameter  $\mu$ .
    const Doub BIG=numeric_limits<Doub>::max();

    Int i,j,iter,status;
    Int m=a.nrows;
    Int n=a.ncols;
    VecDoub y(m),z(n),ax(m),aty(n),rp(m),rd(n),d(n),dx(n),dy(m),dz(n),
        rhs(m),tempm(m),tempn(n);
    NRsparseMat at=a.transpose();   Compute  $\mathbf{A}^T$ .
    ADAT adat(a,at);               Setup for  $\mathbf{A} \cdot \mathbf{D} \cdot \mathbf{A}^T$ , where  $\mathbf{D} = \mathbf{X} \cdot \mathbf{Z}^{-1}$ .
    NRldl solver(adat.ref());       Initialize interface to LDL package.
    solver.order();                AMD ordering and LDL symbolic factorization.
    Doub rpfact=1.0+sqrt(dotprod(b,b)); Compute factors for convergence test.
    Doub rdfact=1.0+sqrt(dotprod(c,c));
    for (j=0;j<n;j++) {            Initial point.
        x[j]=1000.0;
        z[j]=1000.0;
    }
    for (i=0;i<m;i++) {
        y[i]=1000.0;
    }
    Doub normrp_old=BIG;
    Doub normrd_old=BIG;
    cout << setw(4) << "iter" << setw(12) << "Primal obj." << setw(9) <<
        "||r_p||" << setw(13) << "Dual obj." << setw(11) << "||r_d||" <<
        setw(13) << "duality gap" << setw(16) << "normalized gap" << endl;
    cout << scientific << setprecision(4);
    for (iter=0;iter<MAXITS;iter++) { Start of main loop.
        ax=a.ax(x);                Compute normalized residuals  $r_p$  and  $r_d$ .
        for (i=0;i<m;i++)
            rp[i]=ax[i]-b[i];
        Doub normrp=sqrt(dotprod(rp,rp))/rpfact;
        aty=at.ax(y);
        for (j=0;j<n;j++)
            rd[j]=aty[j]+z[j]-c[j];
        Doub normrd=sqrt(dotprod(rd,rd))/rdfact;
        Doub gamma=dotprod(x,z);    Duality gap is  $\mathbf{x} \cdot \mathbf{z}$  for feasible points.
        Doub mu=DELTA*gamma/n;      Choice of  $\mu$ .
        Doub primal_obj=dotprod(c,x); Print current iteration.
```

```

Doub dual_obj=dotprod(b,y);
Doub gamma_norm=gamma/(1.0+abs(primal_obj));
  cout << setw(3) << iter << setw(12) << primal_obj << setw(12) <<
    normrp << setw(12) << dual_obj << setw(12) << normrd << setw(12)
    << gamma << setw(12) << gamma_norm<<endl;
if (normrp < EPS && normrd < EPS && gamma_norm < EPS)
  return status=0;           Optimal solution found.
if (normrp > 1000*normrp_old && normrp > EPS)
  return status=1;           Primal infeasible.
if (normrd > 1000*normrd_old && normrd > EPS)
  return status=2;           Dual infeasible.
for (j=0;j<n;j++)           Compute step directions. First form matrix
  d[j]=x[j]/z[j];            $\mathbf{A} \cdot \mathbf{X} \cdot \mathbf{Z}^{-1} \cdot \mathbf{A}^T$ .
adat.updateD(d);
solver.factorize();           Factorize matrix.
for (j=0;j<n;j++)           Form right-hand side.
  tempn[j]=x[j]-mu/z[j]-d[j]*rd[j];
tempm=a.ax(tempn);
for (i=0;i<m;i++)
  rhs[i]=-rp[i]+tempm[i];
solver.solve(dy,rhs);         Solve for  $dy$ .
tempn=at.ax(dy);             Solve for  $dz$ .
for (j=0;j<n;j++)
  dz[j]=-tempn[j]-rd[j];
for (j=0;j<n;j++)           Solve for  $dx$ .
  dx[j]=-d[j]*dz[j]+mu/z[j]-x[j];
Doub alpha_p=1.0;           Find step length.
for (j=0;j<n;j++)
  if (x[j]+alpha_p*dx[j] < 0.0)
    alpha_p=-x[j]/dx[j];
Doub alpha_d=1.0;
for (j=0;j<n;j++)
  if (z[j]+alpha_d*dz[j] < 0.0)
    alpha_d=-z[j]/dz[j];
alpha_p = MIN(alpha_p*SIGMA,1.0);
alpha_d = MIN(alpha_d*SIGMA,1.0);
for (j=0;j<n;j++) {           Step to new point.
  x[j]+=alpha_p*dx[j];
  z[j]+=alpha_d*dz[j];
}
for (i=0;i<m;i++)
  y[i]+=alpha_d*dy[i];
normrp_old=normrp;           Update norms.
normrd_old=normrd;
}
return status=3;             Maximum iterations exceeded.
}

```

10.11.7 Practical Interior-Point Codes

There are a number of important features that would be needed to turn the above simple implementation into a state-of-the-art code.

- Initial point. Choosing a good starting point cuts down the number of iterations required. A good algorithm is described in [7].
- Preprocessing. As for the simplex method, preprocessing can often reduce the size of the problem.
- Scaling. A badly scaled problem can lead to numerical difficulties.
- Handling bounded variables. Suppose that instead of the requirement $\mathbf{x} \geq 0$ the variables are *bounded*:

$$\mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \quad (10.11.27)$$

Here, for simplicity, we have written the vectors of lower and upper bounds, $\mathbf{l}$ and $\mathbf{u}$, as being of length n . In practice, only some of the variables $\mathbf{x}$ may have bounds. One way to deal with bounds is to add them to the system $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ with slack variables in the usual way. However, this increases the dimension of the matrix $\mathbf{A}$. There is a simpler way to proceed. First, lower bounds of the form $l_j \leq x_j$ can be handled by a simple shift: $x'_j = x_j - l_j \geq 0$. Making this replacement everywhere allows the problem to be solved as before, and then you simply undo the shift to get the solution in terms of the original x_j . So without loss of generality we can assume all the bounds are of the form

$$0 \leq \mathbf{x} \leq \mathbf{u} \quad (10.11.28)$$

If we introduce slack variables $\mathbf{s}$ and dual slack variables $\mathbf{w}$, the optimality conditions are

$$\begin{aligned} \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\ \mathbf{x} + \mathbf{s} &= \mathbf{u} \\ \mathbf{A}^T \cdot \mathbf{y} + \mathbf{z} - \mathbf{w} &= \mathbf{c} \\ \mathbf{X} \cdot \mathbf{Z} \cdot \mathbf{e} &= 0 \\ \mathbf{S} \cdot \mathbf{W} \cdot \mathbf{e} &= 0 \end{aligned} \quad (10.11.29)$$

with $\mathbf{x}$, $\mathbf{s}$, $\mathbf{z}$, and $\mathbf{w}$ all nonnegative. It is simple to change the right-hand sides of the last two equations in (10.11.29) to $\tau \mathbf{e}$ and apply Newton's method as for equation (10.11.11). You find that the equations to be solved are very similar in form to equations (10.11.23) – (10.11.25).

- **Predictor-corrector.** Most of the time spent in an iteration goes into the factorization of the matrix in the normal equations. Given the factorization, the solve step is relatively cheap. Mehotra's predictor-corrector method [7] takes advantage of this by using an extra solve step per iteration to improve the overall efficiency of the algorithm.

Recall that in going from equation (10.11.19) to equation (10.11.20) we dropped the term $\Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e}$. The idea in Mehotra's method is to first take a predictor step that solves equation (10.11.20), but with the τ term omitted. The values of $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{z})$ obtained are used to estimate $\Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e}$. Then the corrector step solves for an additional set $(\Delta \mathbf{x}, \Delta \mathbf{y}, \Delta \mathbf{z})$ from equation (10.11.20) with the right-hand side replaced by

$$\begin{bmatrix} 0 \\ 0 \\ \tau \mathbf{e} - \Delta \mathbf{X} \cdot \Delta \mathbf{Z} \cdot \mathbf{e} \end{bmatrix} \quad (10.11.30)$$

The value of τ in equation (10.11.30) is set differently from equation (10.11.12). First $\hat{\mu}$ is computed using the predictor step:

$$\hat{\mu} = (\mathbf{x} + \alpha_p \Delta \mathbf{x}) \cdot (\mathbf{z} + \alpha_d \Delta \mathbf{z}) / n \quad (10.11.31)$$

Here α_p and α_d are the largest values that maintain nonnegativity, but no larger than unity. (No safety factor is used.) Then τ is set as

$$\tau = \left(\frac{\hat{\mu}}{\mu} \right)^2 \hat{\mu} \quad (10.11.32)$$

where μ is computed using the starting values of $\mathbf{x}$ and $\mathbf{z}$ as in equation (10.11.12). This heuristic choice makes τ small when the predictor step gives a large decrease in complementarity, and large otherwise.

The total step is the sum of the predictor and corrector steps. The cutdown factor from α_p or α_d equal to unity is calculated from a heuristic procedure described in [7] or [8].

Gondzio [9,10] has developed an extension to the predictor-corrector algorithm that incorporates higher-order corrections when they can improve the efficiency.

- Better sparse matrix algebra. While AMD is a good general-purpose choice for an ordering algorithm, LDL is a good but basic sparse Cholesky routine, chosen mainly for its simplicity and availability. More powerful algorithms are known and are starting to become publicly available.

One of the problems with the normal equations is that the matrix can be quite dense, even when $\mathbf{A}$ itself is rather sparse. This has motivated algorithms that solve the augmented equations (10.11.22) directly. On some problems, this leads to significant savings. A good implementation will provide both alternatives.

Solving the equations can become numerically delicate, especially as the optimal point is approached. Good implementations will use some form of iterative refinement to preserve accuracy.

- Crossover to the simplex method. Often the convergence of the interior-point algorithm slows down near the optimal point. By switching to a simplex method with a basis that is presumably close to optimal, one can get rapid convergence to the answer. This feature has the additional benefit that the optimal point is given in terms of basis vectors, whereas the interior-point solution never actually attains zeros for any $\mathbf{x}$'s. Some kinds of post-analysis need the actual basis.

Interestingly, using a previously-found solution as the initial point for a “nearby” problem seldom helps much for interior-point methods. The reason is that interior-point methods don’t make good progress from a point near the boundary. The simplex method, by contrast, generally converges much more rapidly with a “warm start.” A good strategy for solving a sequence of closely related problems is therefore interior-point with crossover to an optimal basis for the first one, then simplex with a warm start for the remainder.

There are several codes that are free for noncommercial use and that give full implementations of interior-point methods. We particularly like PCx (in C with Fortran sparse algebra routines) [11] and HOPDM (in Fortran) [10]. For a discussion of more options, including commercial codes, see [12].

So which should you use: a simplex or an interior-point code? If you have only our codes, use the interior-point one. If you have a production implementation of either algorithm, it will probably suffice. If you are solving many large problems, however, you should have both so you can use the optimum algorithm in each case. If you are solving a large problem for the first time, there is a lot to be said for using an interior-point code. There are fewer choices to make to get almost optimal performance. By contrast, finding which particular choices of components in a simplex method give optimal performance can involve a lot of experimentation. Your first try will usually not be as good as a default interior-point code. And often the